1.Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.
Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

**Experiment 1 — Client-Server Using Named Pipes (FIFO)**

**Part A — Create the Server**

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define FIFO_NAME "client_to_server"

int main()
{
    char buffer[100];
    int fd;

    /* Create named pipe */
    mkfifo(FIFO_NAME, 0666);

    printf("Server started...\n");
    printf("Waiting for client message...\n");

    /* Open FIFO for reading */
    fd = open(FIFO_NAME, O_RDONLY);

    if (fd == -1)
    {
        perror("open");
        exit(1);
    }

    while (1)
    {
        int n = read(fd, buffer, sizeof(buffer) - 1);

        if (n > 0)
        {
            buffer[n] = '\0';

            printf("Client: %s", buffer);

            if (strncmp(buffer, "exit", 4) == 0)
            {
                printf("Server shutting down...\n");
                break;
            }

            printf("Server: Message processed successfully.\n");
        }
    }

    close(fd);
    unlink(FIFO_NAME);

    return 0;
}
```

**Part B — Create the Client**

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define FIFO_NAME "client_to_server"

int main()
{
    char message[100];
    int fd;

    /* Open FIFO for writing */
    fd = open(FIFO_NAME, O_WRONLY);

    if (fd == -1)
    {
        perror("open");
        exit(1);
    }

    printf("Connected to server.\n");

    while (1)
    {
        printf("Enter message: ");
        fgets(message, sizeof(message), stdin);

        write(fd, message, strlen(message));

        if (strncmp(message, "exit", 4) == 0)
        {
            break;
        }
    }

    close(fd);

    return 0;
}
```

Part C — Run the Server

```
adithya@LAPTOP-07A78KDI:~/OS$ nano fifo_server.c
adithya@LAPTOP-07A78KDI:~/OS$ gcc fifo_server.c -o fifo_server
adithya@LAPTOP-07A78KDI:~/OS$ ./fifo_server
Server started...
Waiting for client message...
```

Part D — Run the Client

```
adithya@LAPTOP-07A78KDI:~/OS$ gcc fifo_client.c -o fifo_client
adithya@LAPTOP-07A78KDI:~/OS$ ./fifo_client
Connected to server.
Enter message: Hello Server
```

```
adithya@LAPTOP-07A78KDI:~/OS$ ./fifo_client
Connected to server.
Enter message: Hello Server
Enter message: exit
adithya@LAPTOP-07A78KDI:~/OS$
```

Server:

```
adithya@LAPTOP-07A78KDI:~/OS$ ./fifo_server
Server started...
Waiting for client message...
Client: Hello Server
Server: Message processed successfully.
```

Multiple Clients

From Clients Terminal:

```
adithya@LAPTOP-07A78KDI:~/OS$ ./fifo_client
Connected to server.
Enter message: Client 1 message
Enter message:
```

```
adithya@LAPTOP-07A78KDI:~/OS$ ./fifo_client
Connected to server.
Enter message: Client 2 message
Enter message:
```

From Server Terminal:

```
adithya@LAPTOP-07A78KDI:~/OS$ ./fifo_server
Server started...
Waiting for client message...
Client: Client 1 message
Server: Message processed successfully.
Client: Client 2 message
Server: Message processed successfully.
```

Experiment 2 — POSIX Signal Handling

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int signal)
{
    if (signal == SIGINT)
    {
        printf("\nReceived SIGINT (Ctrl+C)\n");
    }
    else if (signal == SIGTERM)
    {
        printf("\nReceived SIGTERM\n");
    }
    else if (signal == SIGUSR1)
    {
        printf("\nReceived SIGUSR1\n");
    }
}

int main()
{
    printf("Process ID: %d\n", getpid());

    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Signal handler program started...\n");
    printf("Waiting for signals...\n");

    while (1)
    {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}
```

```
adithya@LAPTOP-07A78KDI:~/OS$ nano signal_handler.c
adithya@LAPTOP-07A78KDI:~/OS$ nano signal_handler.c
adithya@LAPTOP-07A78KDI:~/OS$ gcc signal_handler.c -o signal_handler
adithya@LAPTOP-07A78KDI:~/OS$ ./signal_handler
Process ID: 1382
Signal handler program started...
Waiting for signals...
Program is running...
Program is running...
Program is running...
2458
Program is running...
Program is running...
Ctrl + CProgram is running...
cProgram is running...
^C
Received SIGINT (Ctrl+C)
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^C
Received SIGINT (Ctrl+C)
Program is running...
^C
Received SIGINT (Ctrl+C)
Program is running...
Program is running...
Program is running...
kill -SIGUSR1 2458
```